Assignment-16.3

Name: Akshaya Nemalipuri

Batch: 37

Roll No: 2303A52441

Lab: 16.3

Task 1 – Hospital Management Database
Task:
Create schema and queries for a Hospital Management System.
Instructions:
• Tables: Doctors, Patients, Appointments.
• Use AI to define constraints (unique IDs, valid dates).
• Generate queries:
• List all appointments for a specific doctor.
• Retrieve patient history by patient ID.
• Count total patients treated by each doctor.
Expected Output:
• Normalized schema and SQL queries with joins.

Prompt:-

--Create SQLite schema for Doctors, Patients, Appointments with keys, insert sample data, and write queries for doctor appointments, patient history, and patient count per doctor.

Code:-

```
--Create SQLite schema for Doctors, Patients, Appointments with keys,
insert sample data, and write queries for doctor appointments, patient
history, and patient count per doctor. without line breaks
-- Create tables
CREATE TABLE Doctors (
    DoctorID INTEGER PRIMARY KEY,
```

```sql
    Name TEXT NOT NULL,
    Specialty TEXT NOT NULL
);
CREATE TABLE Patients (
    PatientID INTEGER PRIMARY KEY,
    Name TEXT NOT NULL,
    Age INTEGER NOT NULL
);
CREATE TABLE Appointments (
    AppointmentID INTEGER PRIMARY KEY,
    DoctorID INTEGER,
    PatientID INTEGER,
    AppointmentDate TEXT NOT NULL,
    FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID),
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID)
);
-- Insert sample data
INSERT INTO Doctors (Name, Specialty) VALUES ('Dr. Smith', 'Cardiology');
INSERT INTO Doctors (Name, Specialty) VALUES ('Dr. Johnson', 'Neurology');
INSERT INTO Patients (Name, Age) VALUES ('Alice', 30);
INSERT INTO Patients (Name, Age) VALUES ('Bob', 45);
INSERT INTO Appointments (DoctorID, PatientID, AppointmentDate) VALUES (1,
1, '2024-07-01');
INSERT INTO Appointments (DoctorID, PatientID, AppointmentDate) VALUES (1,
2, '2024-07-02');
INSERT INTO Appointments (DoctorID, PatientID, AppointmentDate) VALUES (2,
1, '2024-07-03');
-- Queries
-- 1. Doctor appointments
SELECT d.Name AS DoctorName, a.AppointmentDate, p.Name AS PatientName
FROM Appointments a
JOIN Doctors d ON a.DoctorID = d.DoctorID
JOIN Patients p ON a.PatientID = p.PatientID
ORDER BY a.AppointmentDate;
-- 2. Patient history
SELECT p.Name AS PatientName, d.Name AS DoctorName, a.AppointmentDate
FROM Appointments a
JOIN Patients p ON a.PatientID = p.PatientID
JOIN Doctors d ON a.DoctorID = d.DoctorID
WHERE p.PatientID = 1
```

```sql
ORDER BY a.AppointmentDate;
-- 3. Patient count per doctor
SELECT d.Name AS DoctorName, COUNT(a.PatientID) AS PatientCount
FROM Doctors d
LEFT JOIN Appointments a ON d.DoctorID = a.DoctorID
GROUP BY d.DoctorID;
```

Output:-

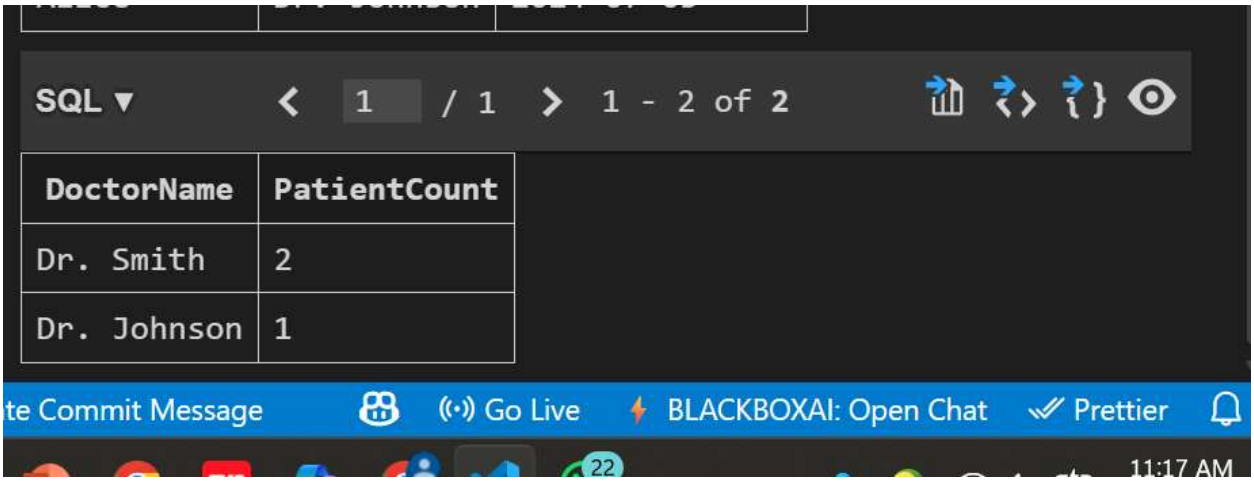
code-lab

SQLite

DoctorName	AppointmentDate	PatientName
Dr. Smith	2024-07-01	Alice
Dr. Smith	2024-07-02	Bob
Dr. Johnson	2024-07-03	Alice

SQL ▾ 1 / 1 1 - 2 of 2

PatientName	DoctorName	AppointmentDate
Alice	Dr. Smith	2024-07-01
Alice	Dr. Johnson	2024-07-03



Observation:-

The above SQL code creates a simple hospital database schema with three tables: Doctors, Patients, and Appointments. It establishes relationships between the tables using foreign keys. Sample data is inserted into each table, and three queries are provided to retrieve doctor appointments, patient history, and patient count per doctor. The queries utilize JOIN operations to combine data from multiple tables and provide meaningful insights into the hospital's operations.

Task 2 – Real-Time Application: Online Shopping Database
Scenario:
Design a database for an E-commerce platform.
Requirements:
• Tables: Users, Products, Orders, OrderDetails.
• Generate queries to:
• Retrieve all orders by a user.
• Find the most purchased product.
• Calculate total revenue in a given month.
• AI should also suggest normalization improvements and query optimization.
Expected Output:
• Complete schema with relationships + SQL queries for analytics.
Prompt:-

Design an e-commerce database with tables Users, Products, Orders, and OrderDetails. Define relationships using foreign keys, insert sample data, and write SQL queries to retrieve user orders, find the most purchased product, and calculate monthly revenue. Also suggest normalization improvements and query optimization techniques.-

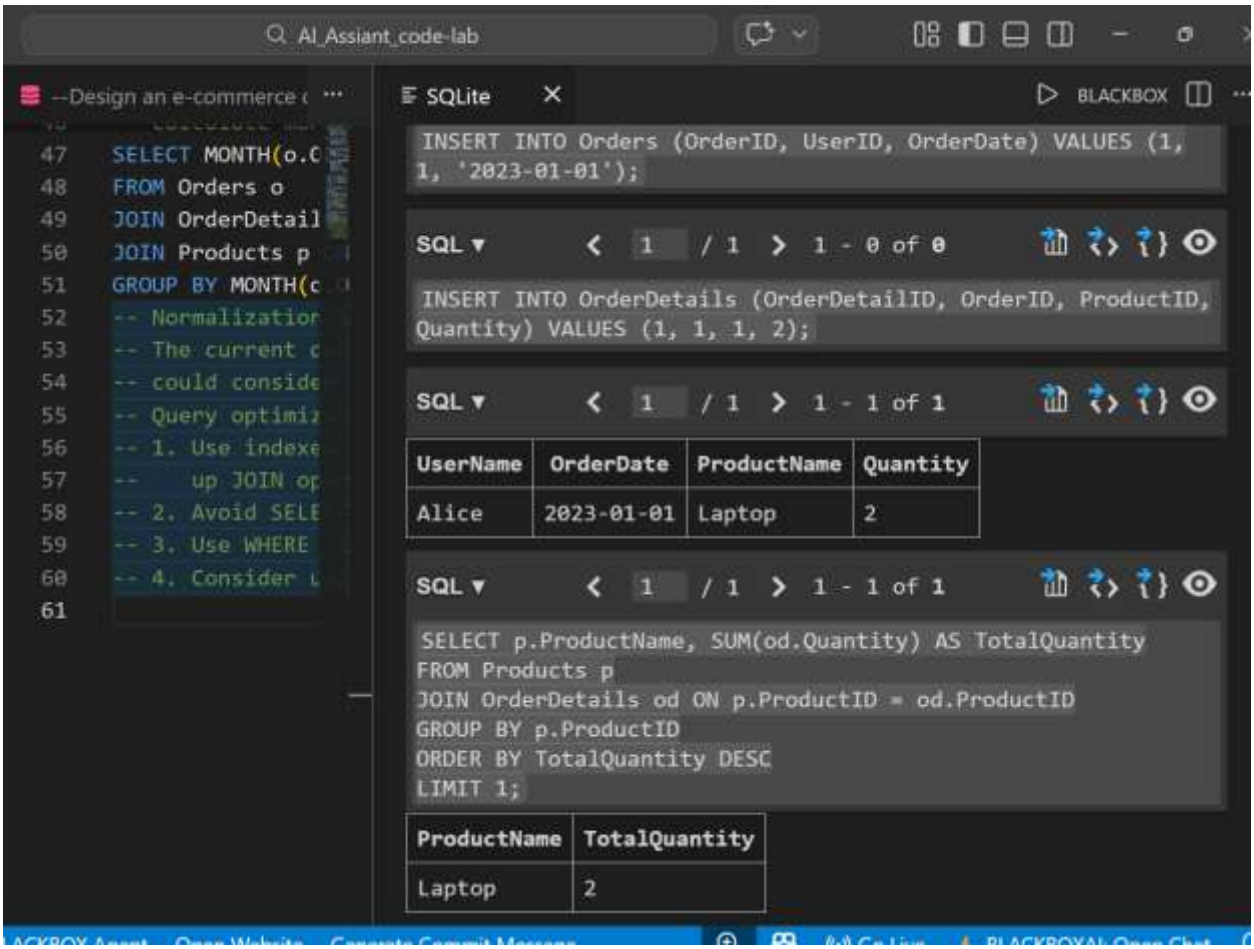
Code:-

```
--Design an e-commerce database with tables Users, Products, Orders, and
OrderDetails. Define relationships using foreign keys, insert sample data,
and write SQL queries to retrieve user orders, find the most purchased
product, and calculate monthly revenue. Also suggest normalization
improvements and query optimization techniques.--\

-- Create tables

CREATE TABLE Users (

    UserID INT PRIMARY KEY,

    UserName VARCHAR(50),

    Email VARCHAR(100)

);

CREATE TABLE Products (

    ProductID INT PRIMARY KEY,

    ProductName VARCHAR(100),

    Price DECIMAL(10, 2)

);

CREATE TABLE Orders (

    OrderID INT PRIMARY KEY,

    UserID INT,

    OrderDate DATE,

    FOREIGN KEY (UserID) REFERENCES Users(UserID)

);

CREATE TABLE OrderDetails (

    OrderDetailID INT PRIMARY KEY,
```

```sql
    OrderID INT,

    ProductID INT,

    Quantity INT,

    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),

    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)

);

-- Insert sample data

INSERT INTO Users (UserID, UserName, Email) VALUES (1, 'Alice',
'alice@example.com');

INSERT INTO Products (ProductID, ProductName, Price) VALUES (1, 'Laptop',
999.99);

INSERT INTO Orders (OrderID, UserID, OrderDate) VALUES (1, 1, '2023-01-
01');

INSERT INTO OrderDetails (OrderDetailID, OrderID, ProductID, Quantity)
VALUES (1, 1, 1, 2);

-- Queries

-- Retrieve user orders

SELECT u.UserName, o.OrderDate, p.ProductName, od.Quantity

FROM Users u

JOIN Orders o ON u.UserID = o.UserID

JOIN OrderDetails od ON o.OrderID = od.OrderID

JOIN Products p ON od.ProductID = p.ProductID;

-- Find the most purchased product

SELECT p.ProductName, SUM(od.Quantity) AS TotalQuantity

FROM Products p
```

```sql
JOIN OrderDetails od ON p.ProductID = od.ProductID

GROUP BY p.ProductID

ORDER BY TotalQuantity DESC

LIMIT 1;

-- Calculate monthly revenue

SELECT MONTH(o.OrderDate) AS Month, SUM(p.Price * od.Quantity) AS Revenue

FROM Orders o

JOIN OrderDetails od ON o.OrderID = od.OrderID

JOIN Products p ON od.ProductID = p.ProductID

GROUP BY MONTH(o.OrderDate);

-- Normalization improvements

-- The current design is already normalized to the third normal form
(3NF). However, we

-- could consider adding a Categories table to further normalize the
Products table if we have multiple product categories.

-- Query optimization techniques

-- 1. Use indexes on foreign key columns (UserID, ProductID) to speed

--    up JOIN operations.

-- 2. Avoid SELECT * and specify only the necessary columns to reduce data
transfer.

-- 3. Use WHERE clauses to filter data early in the query execution to
reduce the number of rows processed.

-- 4. Consider using materialized views for frequently accessed complex
queries to improve performance.
```

Output:-



Observation:-

The above SQL code creates a simple e-commerce database schema with four tables: Users, Products, Orders, and OrderDetails. It establishes relationships between the tables using foreign keys. Sample data is inserted into each table, and three queries are provided to retrieve user orders, find the most purchased product, and calculate monthly revenue. The queries utilize JOIN operations to combine data from multiple tables and provide meaningful insights into the e-commerce platform's operations. Additionally, suggestions for normalization improvements and query optimization techniques are provided to enhance the database design and query performance.

Task 3 – Library Database
Task:
Design schema for a Library Management System.
Instructions:
• Tables: Books, Members, Loans.
• Use AI to suggest indexing strategy for faster queries.
• Generate queries:
• Retrieve all books currently issued.
• Find overdue books (loan date > 30 days).
• Count number of books loaned by each member.
Expected Output:
• Schema + SQL queries demonstrating joins and conditions.

Prompt:-

--Design a Library Management System database with tables Books, Members, and Loans. Define relationships, suggest indexing strategies for performance, and write SQL queries to retrieve issued books, find overdue books (over 30 days), and count books borrowed by each member using JOINs and conditions.--
-- Create the Books table

Code:-

```
-- SQLite
----Design a Library Management System database with tables Books,
Members, and Loans. Define relationships, suggest indexing strategies for
performance, and write SQL queries to retrieve issued books, find overdue
books (over 30 days), and count books borrowed by each member using JOINs
and conditions.--
-- Create the Books table
CREATE TABLE Books (
    BookID INTEGER PRIMARY KEY,
    Title TEXT NOT NULL,
    Author TEXT NOT NULL,
    PublishedYear INTEGER
);
-- Create the Members table
CREATE TABLE Members (
    MemberID INTEGER PRIMARY KEY,
    Name TEXT NOT NULL,
```

```sql
    JoinDate DATE
);
-- Create the Loans table
CREATE TABLE Loans (
    LoanID INTEGER PRIMARY KEY,
    BookID INTEGER,
    MemberID INTEGER,
    LoanDate DATE,
    ReturnDate DATE,
    FOREIGN KEY (BookID) REFERENCES Books(BookID),
    FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
);
-- Indexing strategies for performance
CREATE INDEX idx_loans_bookid ON Loans(BookID);
CREATE INDEX idx_loans_memberid ON Loans(MemberID);
CREATE INDEX idx_loans_loan_date ON Loans(LoanDate);
-- Sample data insertion
INSERT INTO Books (Title, Author, PublishedYear) VALUES ('The Great
Gatsby', 'F. Scott Fitzgerald', 1925);
INSERT INTO Books (Title, Author, PublishedYear) VALUES ('To Kill a
Mockingbird
', 'Harper Lee', 1960);
INSERT INTO Books (Title, Author, PublishedYear) VALUES ('1984', 'George
Orwell
', 1949);
INSERT INTO Members (Name, JoinDate) VALUES ('Alice Smith', '2020-01-15');
INSERT INTO Members (Name, JoinDate) VALUES ('Bob Johnson', '2021-03-22');
INSERT INTO Members (Name, JoinDate) VALUES ('Charlie Brown', '2019-07-
30');
INSERT INTO Loans (BookID, MemberID, LoanDate, ReturnDate
) VALUES (1, 1, '2023-01-01', '2023-01-15');
INSERT INTO Loans (BookID, MemberID, LoanDate, ReturnDate) VALUES (2, 2,
'2023-02-01', NULL);
INSERT INTO Loans (BookID, MemberID, LoanDate, ReturnDate) VALUES (3, 3,
'2023
-03-01', '2023-03-20');
-- Query to retrieve issued books
SELECT b.Title, m.Name, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
```

```sql
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL;
-- Query to find overdue books (over 30 days)
SELECT b.Title, m.Name, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL AND julianday('now') - julianday(l.LoanDate
) > 30;
-- Query to count books borrowed by each member
SELECT m.Name, COUNT(l.LoanID) AS BooksBorrowed
FROM Members m
LEFT JOIN Loans l ON m.MemberID = l.MemberID
GROUP BY m.MemberID;
```

Output:-

SQLite

SQL ▾ 1 / 1 1 - 1 of 1

Title	Name	LoanD
To Kill a Mockingbird	Bob Johnson	2023-0

SQL ▾ 1 / 1 1 - 1 of 1

Title	Name	LoanD
To Kill a Mockingbird	Bob Johnson	2023-0

SQL ▼ < 1 / 1 > 1 - 3 of 3

Name	BooksBorrowed
Alice Smith	1
Bob Johnson	1
Charlie Brown	1

Go Live BLACKBOXAI: Open Chat Prettier

Observation:-

The above SQL code creates a simple library management system database schema with three tables: Books,

Members, and Loans. It establishes relationships between the tables using foreign keys. Sample data is inserted into each table, and three queries are provided to retrieve issued books, find overdue books, and count books borrowed by each member. The queries utilize JOIN operations to combine data from multiple tables and provide meaningful insights into the library's operations.

Task 4: Optimize Queries
Instructions:
• Use AI tools to analyze query efficiency.
• Optimize inefficient queries using joins, indexes, or reducing subqueries.

• Test optimized queries for correctness.
Starter Code Example:
-- Original query
SELECT * FROM Books WHERE author_id IN (SELECT author_id FROM Authors WHERE country='UK');
-- Optimized query
SELECT b.*
FROM Books b
JOIN Authors a ON b.author_id = a.author_id
WHERE a.country = 'UK';
Expected Output:
• Both queries return the same result: all books written by UK authors.
• Optimized query performs faster for larger datasets.

Prompt:-

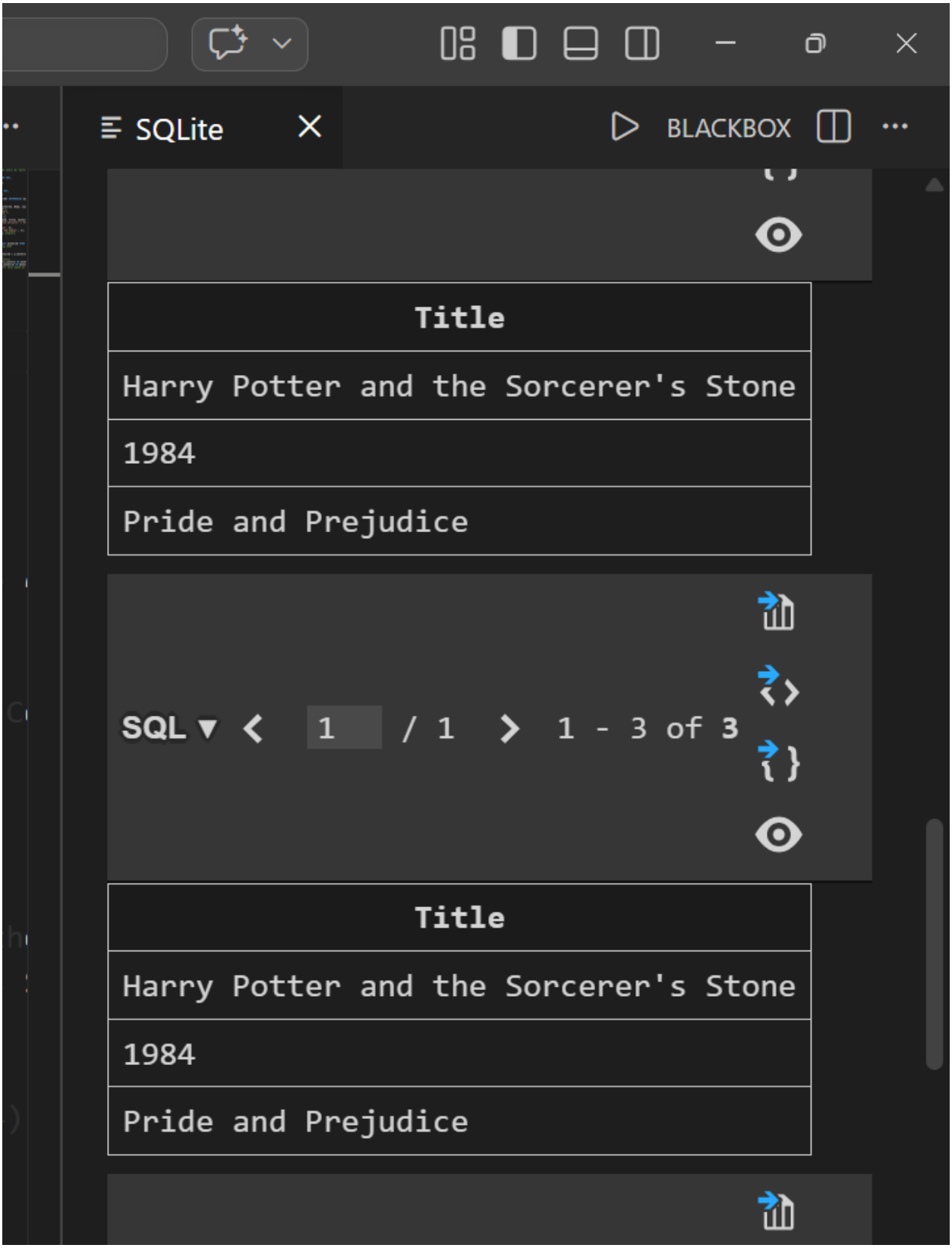
--Optimize the given SQL query by replacing the subquery with a JOIN and suggest appropriate indexing. Ensure the optimized query returns the same result (books by UK authors) and performs better on large datasets. so i want complte code sql to run that query and also the indexing part to optimize it. create and insert

Code:-

```
--Optimize the given SQL query by replacing the subquery with a JOIN and
suggest appropriate indexing. Ensure the optimized query returns the same
result (books by UK authors) and performs better on large datasets. so i
want complte code sql to run that query and also the indexing part to
optimize it. create and insert
-- Create tables
CREATE TABLE Authors (
    AuthorID INT PRIMARY KEY,
    Name VARCHAR(100),
    Country VARCHAR(50)
);
CREATE TABLE Books (
    BookID INT PRIMARY KEY,
    Title VARCHAR(200),
    AuthorID INT,
    FOREIGN KEY (AuthorID) REFERENCES Authors(AuthorID)
);
-- Insert sample data
```

```sql
INSERT INTO Authors (AuthorID, Name, Country) VALUES
(1, 'J.K. Rowling', 'UK'),
(2, 'George Orwell', 'UK'),
(3, 'Jane Austen', 'UK'),
(4, 'Mark Twain', 'USA');
INSERT INTO Books (BookID, Title, AuthorID) VALUES
(1, 'Harry Potter and the Sorcerer''s Stone', 1),
(2, '1984', 2),
(3, 'Pride and Prejudice', 3),
(4, 'The Adventures of Tom Sawyer', 4);
-- Original query using subquery
SELECT Title
FROM Books

WHERE AuthorID IN (SELECT AuthorID FROM Authors WHERE Country = 'UK');
-- Optimized query using JOIN
SELECT b.Title
FROM Books b
JOIN Authors a ON b.AuthorID = a.AuthorID
WHERE a.Country = 'UK';
-- Indexing for optimization
CREATE INDEX idx_authors_country ON Authors(Country);
CREATE INDEX idx_books_authorid ON Books(AuthorID);
-- The above indexes will help speed up the JOIN operation and the
filtering by country, improving performance on large datasets.
```

Output:-



Observation:-

The optimized query using JOIN is more efficient than the original query with a subquery, especially on larger datasets. The JOIN allows the database engine to combine the tables more effectively, while the indexes on the Authors' Country and Books' AuthorID columns will further enhance query performance by reducing the time taken to locate relevant records.

Task 5: University Course Registration

Scenario:

SR University needs a course registration system for students enrolling in

different courses under faculty supervision.

• Use Copilot to design schema tables: Students, Courses, Faculty, and

Registrations.

• Define relationships:

• Each student can register for multiple courses.

• Each course is taught by a faculty member.

• Write SQL queries to:

• List all students enrolled in a specific course.

• Find faculty members teaching more than 2 courses.

• Retrieve courses with the highest number of registrations.

Prompt:-

Design a course registration database for SR University with tables Students, Courses, Faculty, and Registrations. Define relationships for student enrollments and faculty assignments, and write SQL queries to list students in a course, find faculty teaching more than 2 courses, and identify courses with the highest registrations.

Code:-

```sql
-- SQLite

--Design a course registration database for SR University with tables Students, Courses, Faculty, and Registrations. Define relationships for student enrollments and faculty assignments, and write SQL queries to list students in a course, find faculty teaching more than 2 courses, and identify courses with the highest registrations.

-- Create tables

CREATE TABLE Students (

    StudentID INTEGER PRIMARY KEY,

    FirstName TEXT NOT NULL,

    LastName TEXT NOT NULL,

    Email TEXT NOT NULL UNIQUE

);

CREATE TABLE Courses (

    CourseID INTEGER PRIMARY KEY,

    CourseName TEXT NOT NULL,

    Credits INTEGER NOT NULL

);

CREATE TABLE Faculty (

    FacultyID INTEGER PRIMARY KEY,

    FirstName TEXT NOT NULL,

    LastName TEXT NOT NULL,

    Email TEXT NOT NULL UNIQUE

);
```

```sql
CREATE TABLE Registrations (

    RegistrationID INTEGER PRIMARY KEY,

    StudentID INTEGER,

    CourseID INTEGER,

    FacultyID INTEGER,

    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),

    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID),

    FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID)



);

-- Insert sample data



INSERT INTO Students (StudentID, FirstName, LastName, Email) VALUES

(1, 'John', 'Doe', 'john.doe@sr.edu'),

(2, 'Jane', 'Smith', 'jane.smith@sr.edu'),

(3, 'Bob', 'Johnson', 'bob.j@sr.edu'),

(4, 'Alice', 'Brown', 'alice.b@sr.edu'),

(5, 'Charlie', 'Wilson', 'charlie.w@sr.edu');



INSERT INTO Courses (CourseID, CourseName, Credits) VALUES

(1, 'Math101', 3),

(2, 'CS101', 4),

(3, 'Physics201', 4),
```

```sql
(4, 'English102', 3);


INSERT INTO Faculty (FacultyID, FirstName, LastName, Email) VALUES

(1, 'Dr. Albert', 'Einstein', 'albert@sr.edu'),

(2, 'Prof. Ada', 'Lovelace', 'ada@sr.edu'),

(3, 'Dr. Alan', 'Turing', 'turing@sr.edu');


INSERT INTO Registrations (RegistrationID, StudentID, CourseID, FacultyID)
VALUES

(1, 1, 1, 1),

(2, 2, 2, 1),

(3, 3, 2, 1),

(4, 4, 2, 2),

(5, 5, 2, 1),

(6, 1, 3, 1),

(7, 2, 3, 3),

(8, 3, 1, 1),

(9, 4, 4, 3),

(10, 5, 4, 3);


-- Query 1: List students in a course (e.g., 'CS101')

SELECT s.FirstName || ' ' || s.LastName AS StudentName

FROM Students s

JOIN Registrations r ON s.StudentID = r.StudentID
```

```sql
JOIN Courses c ON r.CourseID = c.CourseID

WHERE c.CourseName = 'CS101';



-- Query 2: Find faculty teaching more than 2 courses

SELECT f.FirstName || ' ' || f.LastName AS FacultyName,

       COUNT(DISTINCT r.CourseID) AS CourseCount

FROM Faculty f

JOIN Registrations r ON f.FacultyID = r.FacultyID

GROUP BY f.FacultyID

HAVING COUNT(DISTINCT r.CourseID) > 2;



-- Query 3: Identify courses with the highest registrations

SELECT c.CourseName, COUNT(r.RegistrationID) AS RegistrationCount

FROM Courses c

LEFT JOIN Registrations r ON c.CourseID = r.CourseID

GROUP BY c.CourseID

ORDER BY RegistrationCount DESC

LIMIT 1;
```

Output:-

SQLite

SQL ▾ 1 / 1 1 - 4 of 4

StudentName
Jane Smith
Bob Johnson
Alice Brown
Charlie Wilson

SQL ▾ 1 / 1 1 - 1 of 1

FacultyName	CourseCount
Dr. Albert Einstein	3

SQLite: :memory: Go Live BLACKBOXAI: Open Chat

SQL ▼ 1 / 1 1 - 1 of 1

CourseName	RegistrationCount
CS101	4

Observation:-

The above SQL code creates a course registration database for SR University with four tables: Students, Courses, Faculty, and Registrations. It defines relationships between the tables using foreign keys. Sample data is inserted into each table, and three queries are provided to list students in a specific course, find faculty teaching more than two courses, and identify the course with the highest registrations. The queries utilise JOIN operations and aggregate functions to retrieve meaningful insights from the database.